

JavaScript Basics

An introduction to variables and operators

1. What is JavaScript?

JavaScript (often abbreviated JS) is a high-level, interpreted programming language originally created to make web pages interactive. It runs natively inside web browsers, allowing developers to control page behavior, respond to user actions (clicks, typing, scrolling), update content dynamically, and communicate with servers without reloading the page.

While JavaScript started as a browser-only language, it now also runs outside the browser thanks to environments like Node.js, which lets it power servers, command-line tools, and even desktop and mobile applications. Alongside HTML (structure) and CSS (styling), JavaScript is one of the three core technologies of the web, responsible for behavior and interactivity.

Key characteristics of JavaScript:

- Interpreted: code runs directly without a separate compilation step.
- Dynamically typed: variable types are determined automatically at runtime.
- Multi-paradigm: supports procedural, object-oriented, and functional styles.
- Event-driven: it reacts to things happening, like a button click or data arriving.
- Widely supported: it runs in every modern web browser by default.

2. let, const, and var

JavaScript provides three keywords for declaring variables: var, let, and const. All three store data, but they differ in scope, whether they can be reassigned, and how they behave when used before declaration.

var

var is the original way of declaring variables in JavaScript. It is function-scoped and can be re-declared and reassigned.

```
var name = "Ada";  
console.log(name); // Ada  
  
var name = "Grace"; // re-declaring is allowed  
console.log(name); // Grace
```

let

let declares a variable that can be reassigned but not re-declared in the same scope. It is block-scoped, meaning it only exists inside the { } block where it was defined.

```
let age = 25;  
age = 26; // reassignment is fine  
console.log(age); // 26
```

const

const declares a variable whose reference cannot be reassigned after it is set. It is block-scoped like let, and it must be given a value when declared.

```
const pi = 3.14159;
console.log(pi); // 3.14159

pi = 3.15; // Error: Assignment to constant variable.
```

Key differences and when to use each

- Scope: var is function-scoped; let and const are block-scoped (limited to the nearest { }, such as inside an if statement or loop).
- Reassignment: var and let can be reassigned; const cannot be reassigned once set.
- Re-declaration: var can be declared again in the same scope without error; let and const cannot.
- Hoisting behavior: var is hoisted and initialized as undefined; let and const are hoisted but not initialized, so using them before declaration throws an error (the "temporal dead zone").
- Best practice: use const by default, switch to let only when a variable's value needs to change, and avoid var in modern JavaScript.

3. Operators in JavaScript

Operators are symbols that perform actions on values (called operands). Below are four common categories, each with five examples.

3.1 Arithmetic Operators

Arithmetic operators perform mathematical calculations on numeric values.

Operator	Description	Example	Result
+	Addition	5 + 2	7
-	Subtraction	5 - 2	3
*	Multiplication	5 * 2	10
/	Division	5 / 2	2.5
%	Modulus (remainder)	5 % 2	1

3.2 Assignment Operators

Assignment operators assign values to variables, sometimes combined with an arithmetic operation.

Operator	Description	Example	Result
=	Assigns a value	let x = 5;	x is 5
+=	Adds then assigns	x += 3;	x is 8
-=	Subtracts then assigns	x -= 2;	x is 6
*=	Multiplies then assigns	x *= 4;	x is 24

Operator	Description	Example	Result
<code>/=</code>	Divides then assigns	<code>x /= 6;</code>	<code>x</code> is 4

3.3 Comparison Operators

Comparison operators compare two values and return a boolean (true or false).

Operator	Description	Example	Result
<code>==</code>	Equal value (loose)	<code>5 == "5"</code>	true
<code>===</code>	Equal value and type (strict)	<code>5 === "5"</code>	false
<code>!=</code>	Not equal (loose)	<code>5 != 8</code>	true
<code>></code>	Greater than	<code>8 > 5</code>	true
<code><=</code>	Less than or equal to	<code>5 <= 5</code>	true

3.4 Statement (Logical) Operators

Logical operators are used to combine or evaluate conditions, typically inside statements like if. They also return a boolean result.

Operator	Description	Example	Result
<code>&&</code>	Logical AND — true if both sides are true	<code>true && false</code>	false
<code> </code>	Logical OR — true if either side is true	<code>true false</code>	true
<code>!</code>	Logical NOT — reverses the boolean value	<code>!true</code>	false
<code>if</code>	Runs code only if a condition is true	<code>if (5 > 2) { ... }</code>	runs block
<code>else</code>	Runs code when the if condition is false	<code>if (5 < 2) {} else { ... }</code>	runs block